AutoPass-Gen: Scenario Generation for Evaluating Urgency-Aware Passing Decisions in CARLA

Rohan Sachdeva Xinwei Mai Pranav Prabu Rathang Pandit
University of California, San Diego
{rosachdeva, xmai, pprabu, rpandit}@ucsd.edu

1. Problem

Autonomous driving systems are often evaluated on fixed routes, random traffic, or a small set of hand-designed scenarios. These evaluations can miss rare failures because the simulator is not actively searching for situations where a driving policy makes the wrong decision. We focus on one concrete and safety-critical maneuver: **passing a slow lead vehicle**. The ego vehicle must decide whether to wait, pass, or replan while reasoning about front distance, rear traffic, oncoming gaps, visibility, road geometry, sensor reliability, and user urgency.

We study **scenario generation for urgency-aware passing**. Each generated case contains both CARLA world facts, such as route, traffic, occlusion, and weather, and a synthetic travel request, such as "I need to arrive in 10 minutes," The request is not meant to require a full chatbot; instead it gives the agent a structured context from which to infer start, goal, deadline, urgency level, and delay cost. Our goal is to generate CARLA scenarios that stress-test whether a driving system can safely trade off progress and risk. The core research question is: *under what generated conditions does urgency cause a passing policy to become unsafe, over-conservative, or inconsistent?*

2. Motivation

Several autonomous driving projects focus on perception, navigation, or low-level control. Our project is different: we focus on **generation-driven evaluation**. Passing is narrow enough to implement in CARLA but rich enough to expose meaningful failures. A fully conservative agent may never pass and therefore fail time-sensitive tasks; an urgency-driven agent may pass in unsafe gaps. The interesting behavior lies at the boundary.

Xinwei's decision diagram 1 motivates our policy loop. To avoid requiring a real-time chatbot, the Scenario Generator produces a synthetic travel request together with the CARLA scene. A Request/Urgency Interpreter then extracts the start, goal, deadline, urgency level, and delay cost from this request. The ego vehicle follows a normal route and enters the passing loop only when a slow front vehicle blocks progress. The loop repeatedly evaluates the current state, uses tools such as RGB/depth perception, SLAM/map state, segmentation, and safety checking, and then chooses `pass`, `wait`, or `replan`. This gives us a concrete agentic system with tool use, self-correction, and measurable outcomes.

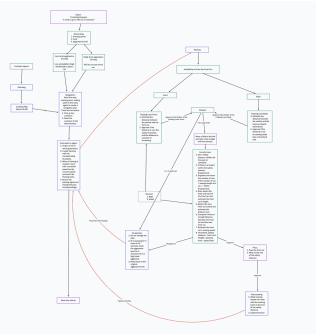

Figure 1. Urgency-Aware Passing Decision Diagram. The diagram shows how a generated travel request and CARLA scene are converted into an urgency-aware passing loop. Blue boxes indicate major stages, green boxes indicate agent reasoning or checking steps, and purple boxes indicate inputs, outputs, or actions.

3. Key Background Literature

We use CARLA [3] because it provides repeatable urban environments, configurable weather, dynamic actors, and access to RGB/depth sensors and simulator state. Prior work on conditional imitation learning [2] and privileged training [1] studies how policies can learn to drive from data or privileged information. However, these works do not directly answer which safety-critical scenarios should be generated to expose policy weaknesses.

Our work is closer to adversarial testing and falsification: rather than only training a policy, we generate situations that reveal failures. The contribution is a compact passing scenario representation, a closed-loop scenario generator, and an interpretable urgency-aware passing policy whose deci-

sions can be audited through distances, gaps, urgency, visibility, and critic approvals.

4. Method Overview

AutoPass-Gen evaluates urgency-aware passing as a closed-loop generation and testing problem. Each trial begins with a structured `ScenarioSpec` containing route information, a synthetic travel request, ego/lead/rear/oncoming traffic, occlusion, weather, and sensor settings. The request is interpreted into a deadline, urgency level, and delay cost. The ego vehicle then follows a route until it encounters a slow lead vehicle, at which point the passing loop begins.

At each timestep, the Perception/Map module estimates the current `PassState`, including front distance, rear distance, oncoming distance, relative speeds, visibility, deadline pressure, and urgency. The Passing Agent proposes one of three actions: `pass`, `wait`, or `replan`. The Safety Checker accepts or rejects passing proposals using time-to-collision, rear-gap, oncoming-gap, front-gap, and visibility constraints. Approved decisions are sent to the Execution module, while rejected decisions are converted into safer wait or replan behavior. The Evaluator records collisions, route completion, time-to-goal, pass attempts, unsafe passes, minimum TTC, and the final failure category.

AutoPass-Gen has three coupled components: a **scenario generator**, a **request/urgency interpreter**, and an **urgency-aware passing policy**. The generator creates difficult passing cases and synthetic travel requests; the interpreter converts those requests into route and urgency variables; and the policy is the system being evaluated during closed-loop driving. The key research idea is that the Scenario Generator is not passive: rollout failures can be used to mutate future scenarios toward narrower gaps, stronger occlusions, tighter deadlines, and more difficult traffic configurations.

4.1. Scenario Representation

We represent each case as a structured specification:

$$\begin{aligned}\texttt{ScenarioSpec} = \{&\texttt{map}, \texttt{route}, \texttt{request}, \texttt{ego},\\ &\texttt{lead}, \texttt{rear}, \texttt{oncoming},\\ &\texttt{occlusion}, \texttt{weather}, \texttt{sensor}\}.\end{aligned}$$

Here, `request` contains a synthetic travel instruction with a start, goal, deadline, and optional natural-language reason for urgency. The generator creates this request but does not directly decide the policy's risk tolerance. Instead, the Request/Urgency Interpreter maps the request to an urgency level and delay cost. The remaining fields define the physical scene: `lead` controls the front vehicle distance, speed, and acceleration; `rear` controls whether lane change is safe from behind; `oncoming` controls the available passing gap; `occlusion` models parked cars, curves, junctions, or limited sight distance; and `weather` changes RGB reliability. This is higher-level than raw CARLA scripts but lower-level than unrestricted natural language, making scenarios easy to instantiate, perturb, log, and compare.

4.2. Request and Urgency Interpretation

A key design choice is that the generator creates the initial facts, while the agentic system performs the interpretation. For example, the generator may create a request such as "drive from A to B and arrive within 10 minutes" together with a CARLA scene containing a slow lead vehicle, rear traffic, and an oncoming gap. The Request/Urgency Interpreter then computes structured variables such as

$$\begin{aligned}&\texttt{start}, \quad \texttt{goal}, \quad \texttt{deadline},\\ &\texttt{urgency_level}, \quad \texttt{delay_cost}.\end{aligned}$$

This avoids requiring a real user-facing chatbot while preserving the agentic reasoning step: urgency is inferred from the generated request rather than directly hard-coded as the final policy decision. During rollout, the agent repeatedly evaluates the current state, proposes `pass`, `wait`, or `replan`, executes the approved action, and then reevaluates the updated state.

4.3. Scenario Generation Loop

The generator samples or mutates `ScenarioSpec` values to create increasingly difficult cases: slow lead vehicles, deceptive oncoming gaps, unsafe rear vehicles, occlusions, weather changes that require sensor-mode adaptation, and deadlines that induce different levels of urgency. After each rollout, the evaluator returns safety and efficiency metrics. Failure cases are fed back into the generator, which increases difficulty by narrowing gaps, increasing occlusion, changing traffic timing, tightening deadlines, or changing the route context. The generator therefore controls the experimental distribution, while the agent still performs the interpretation and decision-making during rollout.

4.4. Urgency-Aware Passing Policy

The ego vehicle follows a CARLA baseline agent until it detects a slow lead vehicle. The passing loop then estimates front distance, rear distance, relative velocity, visibility, and oncoming traffic. It approves a pass only when two conditions hold: (i) the maneuver improves progress under the current urgency, and (ii) geometric and temporal safety margins remain valid. A simplified safety check is

$$d_{\text{gap}} > v_{\text{ego}} t_{\text{pass}} + \frac{1}{2} a_{\text{ego}} t_{\text{pass}}^2 + \Delta,$$

where d_{gap} is available distance before conflict, t_{pass} is estimated passing time, and Δ is a safety buffer. Urgency may lower the threshold for considering a pass, but it cannot override collision risk, insufficient visibility, or unsafe rear traffic.

5. System Architecture

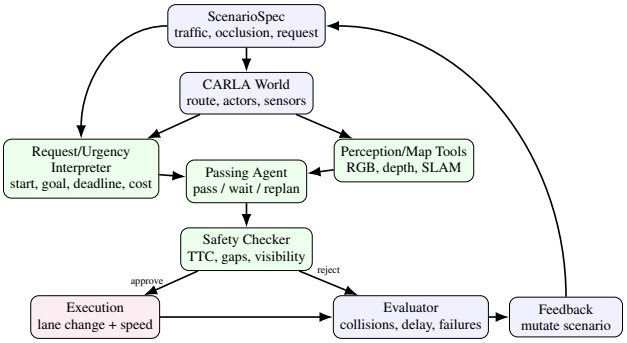

Figure 2. AutoPass-Gen architecture. The main research object is the closed-loop generation and evaluation of passing scenarios; the passing agent provides an interpretable policy to test.

6. Working Prototype Details

We implemented an initial Python prototype of the full AutoPass-Gen architecture. The current prototype is **simulator-independent**: it uses a lightweight kinematic rollout backend rather than CARLA so that we can validate the scenario generation, urgency interpretation, passing policy, safety checking, logging, and evaluation loop before all team members need to run CARLA. This is not intended to replace CARLA; rather, it gives us a stable integration target whose world-state and execution interfaces can be swapped for CARLA actor spawning, sensors, and controls.

The prototype implements the major components in Figure 2. The **Scenario DSL** stores each generated case as a typed `ScenarioSpec`. The **Scenario Generator** samples deadlines, lead vehicle speeds, rear traffic, oncoming gaps, occlusion levels, weather, and sensor modes. The **Request/Urgency Interpreter** converts synthetic travel requests into structured urgency variables. The **Perception/Map Tools** estimate front distance, rear distance, oncoming distance, visibility, and relative speeds using privileged or noisy sensor-like state. The **Passing Agent** proposes `plan`, `wait`, or `replan`. The **Safety Checker** rejects unsafe passes using TTC, rear-gap, oncoming-gap, front-gap, and visibility constraints. The **Execution Agent** updates the ego state with a simple kinematic modl, and the **Evaluator** writes per-timestep traces and aggregate metrics.

The prototype also includes three policy variants: **AutoPass**, our urgency-aware passing policy; **no-pass**, a conservative baseline that never attempts passing; and **aggressive**, a progress-seeking baseline that attempts risky passes. Each rollout produces a JSON trace containing distances, visibility, urgency, decision, risk score, critic approval, and reason strings, as well as a CSV row containing collision status, route completion, time-to-goal, pass attempts, unsafe passes, minimum TTC, and failure type.

Policy	Failures	Avg. time (s)	Pass attempts	Unsafe passes
No-pass	16/50	96.58	0	0
AutoPass	13/50	90.16	19	0
Aggressive	41/50	60.08	154	134

Table 1. Initial local evaluation over 50 generated scenarios. AutoPass attempts selected passes and reduces delay relative to non-pass while avoiding unsafe passes. The aggressive baseline is faster but produces many unsafe passes.

Policy	None	Delay	Missed safe pass	Unsafe pass
No-pass	34	16	0	0
AutoPass	37	11	2	0
Aggressive	9	2	0	39

Table 2. Failure type distribution for the 50-scenario local evaluation. AutoPass avoids unsafe passes while reducing conservative-delay failures relative to no-pass.

The failure taxonomy currently includes `unsafe_pass`, `missed_safe_pass`, `over_conservative_delay`, `sensor_mode_failure`, and `collision`.

7. Initial Results and Observations

We evaluated the local prototype on 50 generated passing scenarios using the three policies described above. These results should be interpreted as an integration and debugging checkpoint rather than a final CARLA evaluation. The purpose is to test whether the full AutoPass-Gen loop produces meaningful safety-efficiency tradeoffs before replacing the lightweight backend with CARLA.

Table 1 shows the intended qualitative behavior. The no-pass baseline never attempts a pass and produces no unsafe-pass failures, but it has the highest average time-to-goal and produces 16 over-conservative delay failures. The aggressive baseline achieves the lowest average time-to-goal, but fails in 41 out of 50 scenarios, mostly through unsafe passes. AutoPass produces an intermediate behavior: it attempts passes selectively, has zero unsafe-pass failures, reduces over-conservative delay failures from 16 to 11 relative to no-pass, and reduces average time-to-goal from 96.58s to 90.16s.

The failure breakdown in Table 2 supports the main project hypothesis. Generated urgent passing scenarios expose the tradeoff between progress and safety: a policy that never passes avoids unsafe behavior but can miss deadlines, while a policy that aggressively pursues progress frequently chooses unsafe gaps. AutoPass lies between these extremes by using urgency to consider passing, while still allowing the Safety Checker to reject proposals with insufficient TTC, poor visibility, unsafe rear traffic, or inadequate oncoming gap.

The trace format was also useful for debugging the prototype. Early local runs revealed two backed issues: on-

coming vehicles were initially counted as collisions even when the ego vehicle was not in the passing lane, and waiting behind a lead vehicle initially lacked safe car-following behavior. We fixed these by making collision checks lane-aware, computing distances from explicit actor positions, and adding adaptive cruise-control-like following behavior for wait and replan actions. These fixes make the local prototype suitable for generating initial metrics and interpretable traces while the next phase connects the same interfaces to CARLA.

8. Representations and Proposed DSL

Generated scenarios will be implemented as compact JSON-like objects:

```
{
  request: {text, start, goal, deadline},
  route: {town, start, goal, lane_type},
  lead: {distance, speed, accel},
  rear: {distance, speed},
  oncoming: {distance, speed, gap_time},
  occlusion: {type, severity, sight_distance},
  weather: {rain, fog, sun_angle},
}
```

The runtime state passed between agents is

$$\texttt{PassState} = \{\texttt{ego}, \texttt{lead}, \texttt{rear}, \texttt{gap}, \texttt{visibility}, \texttt{deadline}, \texttt{urgency}, \texttt{delay_cost}, \texttt{decision}, \texttt{risk}\}$$

Every rollout therefore produces not only a video and metric score, but also a trace explaining why the agent passed, waited, or replanned.

9. Agent Roles

Figure 2 shows the basic architecture for the experiment. The **Scenario Generator** proposes CARLA configurations and synthetic travel requests that stress passing. The **Request/Urgency Interpreter** extracts the route, deadline, urgency level, and delay cost from the generated request and world context. The **Perception/Map Tools** estimate distances, lane geometry, visibility, and traffic context from RGB, depth, SLAM, or simulator state. The **Passing Agent** proposes `pass`, `wait`, or `replan`. The **Safety Checker** rejects unsafe proposals using TTC, rear-gap, front-gap, and visibility constraints. The **Execution Agent** converts approved decisions into CARLA controls. The **Evaluator** logs metrics and sends failures back to the generator.

10. Expected Outcomes

The main expected outcome is a CARLA-backed version of the current AutoPass-Gen loop. The local prototype already implements the structured scenario representation, urgency interpretation, passing/no-passing policies, safety checking, trace logging, and metric evaluation. The remaining work is to replace the lightweight kinematic backend with CARLA actor spawning, map-based routes, sensors, and vehicle controls.

Our next milestone is a minimal CARLA scenario in which a `ScenarioSpec` spawns an ego vehicle, slow lead vehicle, rear vehicle, and oncoming vehicle on a two-lane route. We will first use simulator state for distances and traffic context, then add RGB/depth-based estimates for selected quantities if time permits. The final evaluation will compare AutoPass against the no-pass and aggressive baselines on a small suite of generated CARLA scenarios.

By the final report, we aim to deliver: (1) a structured scenario generator for controlled passing stress tests, (2) an urgency-aware passing baseline with interpretable traces, (3) quantitative comparison against conservative and aggressive baselines, (4) a failure taxonomy covering unsafe pass, missed safe pass, over-conservative delay, sensor-mode failure, and collision, and (5) at least a small number of CARLA-backed examples with screenshots or videos.

11. Scope and Feasibility

The project is scoped around generation-driven evaluation rather than building a full autonomous driving stack from scratch. Our current prototype deliberately separates the scenario, policy, safety, execution, and evaluation interfaces from the simulator backend. This lets us test the full decision loop locally while keeping CARLA integration focused and feasible.

For the next phase, we will use CARLA's built-in agents, traffic manager, and simulator state where needed. The minimum CARLA-backed system only needs to instantiate a small number of passing scenarios, run the pass/wait/replan loop, and log traces using the same `ScenarioSpec` and `PassState` interfaces. If time permits, we will replace privileged state estimates with RGB/depth-based perception for selected quantities such as front distance, visibility, or vehicle presence. This preserves the main research contribution: generated scenarios that expose when urgency-aware passing policies become unsafe, over-conservative, or inconsistent.

12. Team Plan

To complete the project in the remaining weeks, we divide the work by interface boundaries. Rohan will own integration, evaluation, plotting, and report writing, ensuring that each module plugs into the shared `ScenarioSpec` and `PassState` formats. Xinwei will own the passing logic and Safety Checker, aligning the implemented pass/wait/replan loop with the decision diagram and pro-

ducing policy pseudocode for the report. Pranav will own CARLA scenario instantiation, including actor spawning, map and route setup, weather, sensors, and a small number of working CARLA examples. Rathang will own perception/map tools, baseline comparisons, metrics, and summary plots.

The immediate Week 7 target is a minimal CARLA-backed scenario with one ego vehicle, one slow lead vehicle, and one oncoming vehicle. Week 8 will focus on comparisons and failure cases across AutoPass, no-pass, and aggressive policies. Week 9 will focus on final figures, trace examples, screenshots or videos, and report polish.

References

[1] Dian Chen, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Conference on Robot Learning*, 2019. 1

[2] Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *IEEE International Conference on Robotics and Automation*, 2018. 1

[3] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. In *Conference on Robot Learning*, 2017. 1